Assignment 1 Report

Movie Reviews Sentiment Analysis Using Natural Language Processing (NLP)

Course: Natural Language Processing
Assignment: 1 – Dataset Exploration and Text Preprocessing
Project Topic: Movie Reviews Sentiment Analysis
Dataset: IMDB Movie Reviews Dataset
Team Members: Add your team members' names here
Submission Date: 8 November 2026

---

1. Introduction

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables computers to understand and process human language.

The purpose of this project is to prepare movie reviews for sentiment analysis. Sentiment analysis identifies whether a review expresses a positive or negative opinion.

In this assignment, we explore a movie review dataset, examine its text characteristics, compare tokenization methods, and implement a reusable text-cleaning pipeline.

2. Dataset and Environment

2.1 Dataset Information

We used the IMDB Movie Reviews Dataset.

- Dataset source: https://zenodo.org/records/7928582
- Total documents: 50,000
- Positive reviews: 25,000
- Negative reviews: 25,000
- Columns: review and sentiment
- File format: CSV
- Licence: Not specified in the referenced Zenodo record.

The dataset contains movie reviews labelled as positive or negative. The raw dataset is stored locally and is not included in the Git repository.

2.2 Environment and Libraries

The project was developed using Python in Visual Studio Code.

Libraries used:

- Pandas: Loading and processing CSV data.
- NLTK: Porter stemming.
- re: Regular expressions and tokenization.
- unicodedata: Unicode normalization.
- pathlib: Managing file paths.

Dependencies are listed in `requirements.txt`.

2.3 Dataset Loading

The dataset was loaded using Pandas. The loading script checks whether the raw dataset exists, reads the CSV file, standardizes column names, removes missing values, and saves the prepared dataset.

The script is available at:

`src/download_dataset.py`

3. Dataset Exploration

3.1 Basic Statistics

The following statistics were calculated:

Measurement	Result
Total documents	50,000
Average review length	231.16 words
Shortest review	4 words
Longest review	2,470 words
Positive reviews	25,000
Negative reviews	25,000

3.2 Target Distribution

The dataset has a balanced sentiment distribution.

Sentiment	Number of Reviews
Positive	25,000
Negative	25,000

Each class represents 50% of the dataset. This balance is useful because neither sentiment class dominates the dataset.

3.3 Vocabulary Analysis

Vocabulary analysis was performed on the first 1,000 reviews.

Measurement	Result
Sample documents	1,000
Total tokens	239,546
Unique words (types)	17,960
Type-token ratio	0.075

The type-token ratio was calculated by dividing the number of unique words by the total number of tokens.

A low ratio indicates that many words occur repeatedly in the reviews.

3.4 Zipf's Law and Long-Tail Distribution

The most frequent words include:

Word	Frequency
the	13,431
and	6,474
a	6,385
of	5,911
to	5,319
br	4,124
is	4,069
in	3,727
it	3,668

The results show that a small number of words occur very frequently, while many other words occur less frequently.

This pattern is related to Zipf's Law and the long-tail distribution of natural language.

The word `br` appears because HTML line-break tags such as `<br />` are present in the original reviews.

4. Tokenization and Regular Expressions

4.1 Tokenization Methods

Two tokenization methods were tested on the first 1,000 reviews.

Method 1: Whitespace Tokenization

Whitespace tokenization uses the `split()` method to divide text wherever whitespace occurs.

Method 2: Regex-Based Tokenization

Regex tokenization uses the following pattern:

`\b\w+\b`

It extracts individual words and numbers based on word boundaries.

4.2 Tokenization Comparison

Method	Total Tokens
Whitespace tokenization	231,265
Regex tokenization	239,546

Regex tokenization produced more tokens because it separates some punctuation-connected words and contractions.

For example:

- Whitespace: `you'll`
- Regex: `you`, `ll`

Similarly, the hyphenated word `light-hearted` becomes `light` and `hearted`.

4.3 Useful Regular Expressions

Three regular expressions were examined.

1. HTML Tag Removal

Pattern:

`<[^>]+>`

Purpose: Detects HTML tags such as `<br />` so they can be removed from reviews.

2. Punctuation and Special Character Detection

Pattern:

`[^a-zA-Z0-9\s]`

Purpose: Detects characters that are not English letters, numbers, or whitespace.

3. Word Extraction

Pattern:

`\b\w+\b`

Purpose: Extracts individual words and numbers from text.

4.4 Dataset-Specific Tokenization Challenges

Two important challenges were identified:

1. HTML tags: Reviews contain HTML tags such as `<br /><br />`, which can create unwanted tokens if they are not removed.
2. Contractions and hyphenated words: Words such as `you'll` and `light-hearted` may be treated differently by different tokenization methods.

These challenges show why text preprocessing is important before sentiment analysis.

5. Text Cleaning Pipeline

A reusable text-cleaning function was implemented in:

`src/clean_text.py`

The pipeline contains the following steps:

1. Unicode Normalization

NFKC normalization is applied to standardize Unicode characters.

2. HTML Tag Removal

HTML tags are removed using regular expressions.

3. Lowercase Conversion

All text is converted to lowercase so that words with different capitalization are treated consistently.

4. Tokenization

Words and numbers are extracted using a regular expression.

5. Stopword Removal

Common words that provide limited information are removed using a manually defined stopword list.

Negation words such as `not`, `no`, and `never` are deliberately preserved because they can change the meaning of a movie review.

6. Stemming

Porter Stemmer from NLTK is used to reduce words to their stems.

For example, words such as `watching` and `watched` may be reduced to a common stem.

5.1 Cleaning Function

The cleaning function performs all preprocessing operations and returns cleaned text as a single string.

It is reusable because the same function can be applied to every review in the dataset.

5.2 Vocabulary Before and After Cleaning

Vocabulary was measured across the full dataset before and after cleaning.

Measurement	Vocabulary Size
Before cleaning	101,944
After cleaning	70,331
Reduction	31,613

The vocabulary decreased by approximately 31%.

This reduction occurred because HTML tags, punctuation, common stopwords, and different word forms were handled during preprocessing.

5.3 Example of Cleaning

Original review:

```
One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked.
```

Cleaned review:

```
one other review mention after watch just 1 oz episod ll hook
```

The cleaned text contains lowercase tokens, fewer unnecessary words, and stemmed word forms.

6. Output and Reproducibility

The cleaned dataset was successfully saved as:

`data/processed/cleaned_reviews.csv`

The output contains the original review, sentiment label, and an additional `cleaned_review` column.

The project uses separate scripts for dataset preparation and text cleaning.

To reproduce the process:

1. Install Python.
2. Create and activate a virtual environment.
3. Install dependencies using `pip install -r requirements.txt`.
4. Place the original dataset in `data/raw/`.
5. Run `python src/download_dataset.py`.
6. Run `python src/clean_text.py`.

The scripts generate the prepared and cleaned datasets automatically.

7. Conclusion

In this assignment, we explored the IMDB movie review dataset and implemented a complete text preprocessing pipeline.

We examined dataset statistics, sentiment distribution, vocabulary size, and tokenization methods. We also identified common text problems such as HTML tags, contractions, and hyphenated words.

The cleaning pipeline applies Unicode normalization, HTML removal, lowercase conversion, tokenization, stopword removal, and Porter stemming.

The vocabulary decreased from 101,944 to 70,331 unique words after cleaning.

The cleaned dataset is now prepared for the next phase of the sentiment analysis project.

8. Contribution Statement

Team Members and Contributions:

- Member 1: [Add name and actual contribution]
- Member 2: [Add name and actual contribution]
- Member 3: [Add name and actual contribution]

All contributions should reflect the actual work completed by each team member.